

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
ĐỀ TÀI: PHÁT TRIỂN HỆ THỐNG SOẠN
THẢO VĂN BẢN CỘNG TÁC

Giảng viên hướng dẫn	Kim Ngọc Bách
Sinh viên thực hiện	Vũ Minh Dương - B22DCCN174 Tương Thị Chinh-B22DCCN113 Nguyễn Tiến Hải- B22DCCN269
Nhóm báo cáo	20

MỤC LỤC

MỤC LỤC	2
KIẾN TRÚC HỆ THỐNG	4
1.1 Kiến trúc tổng thể	4
1.2 Các thành phần chính	5
1.2.1 Frontend	5
1.2.2 Backend	5
Quản lý phân quyền và Xác thực kết nối (Permission & Authentication Management)	6
Thiết lập và Mở kết nối WebSocket (Connection Management)	6
Tiếp nhận Operation từ Client (Receiving Operations)	6
Điều phối và Gửi Operation vào Message Broker RabbitMQ (Message Queue Ingestion)	7
1.2.3 PostgreSQL	8
1.2.4 Redis	11
1.2.5 RabbitMQ	12
1.2.6 OT Worker	12
1.3 Luồng Chinh Sửa Realtime	12
1.4 Luồng phân quyền	14
1.4.1 Kiến trúc phân quyền 3 tầng (3-Tier Permission Architecture)	14
1.4.2 Cơ chế thu hồi quyền thời gian thực và Ngắt kết nối lập tức (Realtime Revocation & WebSocket Kick)	15
1.5 Docker Deployment	16
MÔ HÌNH ĐỒNG BỘ DỮ LIỆU	16
2.1 Mô hình đồ bộ dữ liệu	16
2.2 Dữ liệu được đồng bộ	17
2.2.3 Chi tiết các thành phần dữ liệu đồng bộ cốt lõi	19
1. Nội dung tài liệu (Document Content)	19
2. Số hiệu phiên bản (Version)	19
3. Đồng hồ Vector (Vector Clock)	20
4. Nhật ký thao tác (Operation Log)	20
5. Trạng thái con trỏ (Cursor Status)	20
6. Trạng thái hiện diện người dùng (User Presence)	21
2.3 Cơ chế chống trùng operation	22
2.4 Luồng Đồng Bộ Khi Người Dùng Chinh Sửa	22
2.5 Vai Trò Của RabbitMQ	23
2.6 Vai Trò Của Redis Và PostgreSQL	23
Redis	23
PostgreSQL	23
2.7 Xử Lý Xung Đột	24

2.8 Mô Hình Nhất Quán	24
2.9 Đồng bộ trạng thái realtime UI	25
2.10 Xử lý offline, reconnect và pending queue	25
2.11 Undo/Redo trong môi trường cộng tác	26
GIAO THỨC TRAO ĐỔI DỮ LIỆU	26
3.1 HTTP REST API	27
3.2. WebSocket Realtime Protocol	28
3.3. Message Operation Từ Client	29
3.4. Message Operation Đã Được Áp Dụng	30
3.5. Message Presence	31
3.6. Message Cursor	32
3.7. Ping/Pong	32
3.8. RabbitMQ Internal Messages	33
THỬ NGHIỆM VÀ ĐÁNH GIÁ	33
4.1 Mục tiêu thử nghiệm	33
4.2 Môi trường thử nghiệm	33
4.3 Đăng Ký Và Đăng Nhập	33
4.4 Quản lý tài liệu	34
4.5 Chức năng phân quyền	34
4.6. WebSocket Và Presence	35
4.7. Chỉnh Sửa Realtime Cơ Bản	35
4.8. Xử Lý Xung Đột Đồng Thời	36
4.9. Chống Trùng Operation	36
4.10. Offline Và Reconnect	36
4.11. Redis Và PostgreSQL	37
4.12. RabbitMQ Và Worker	37
4.13. History, Version, Rollback, Undo/Redo	38
4.14. Đánh Giá Hệ Thống	38
Tính đúng đắn	38
Tính nhất quán	38
Tính chịu lỗi	38
Tính realtime	38
Tính mở rộng	39

KIẾN TRÚC HỆ THỐNG

1.1 Kiến trúc tổng thể

Hệ thống được xây dựng theo mô hình kiến trúc phân tán nhiều tầng (multi-tier distributed architecture) nhằm hỗ trợ soạn thảo tài liệu cộng tác theo thời gian thực.

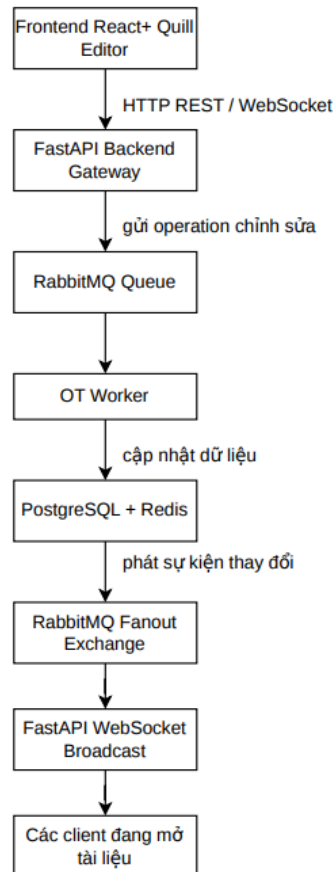
Kiến trúc bao gồm các thành phần chính:

- Frontend React + ReactQuill
- Backend Gateway sử dụng FastAPI
- PostgreSQL lưu trữ dữ liệu bền vững
- RabbitMQ message broker
- OT Worker xử lý đồng bộ dữ liệu và xung đột chỉnh sửa

Luồng xử lý được thiết kế theo hướng bất đồng bộ (asynchronous processing), trong đó Backend Gateway chỉ tiếp nhận kết nối và chuyển operation vào hàng đợi RabbitMQ, còn việc xử lý đồng bộ dữ liệu sẽ do OT Worker đảm nhiệm.

Kiến trúc này giúp:

- giảm tải cho WebSocket Gateway,
- tránh race condition khi nhiều người chỉnh sửa cùng lúc,
- tăng khả năng mở rộng hệ thống
- hỗ trợ xử lý realtime ổn định.



1.2 Các thành phần chính

1.2.1 Frontend

Frontend nằm trong thư mục frontend/src, được xây dựng bằng React và ReactQuill.

Frontend chịu trách nhiệm:

- hiển thị giao diện editor,
- gửi thao tác chỉnh sửa realtime,
- nhận dữ liệu đồng bộ từ server,
- hiển thị danh sách người dùng online,
- cập nhật cursor realtime.

Khi người dùng chỉnh sửa nội dung:

1. ReactQuill sinh ra Quill Delta,
2. Frontend đóng gói thành operation,
3. gửi operation qua WebSocket tới backend.

Frontend không tự quyết định trạng thái cuối cùng của tài liệu mà luôn chờ server xác nhận.

Frontend đảm nhiệm các xử lý realtime quan trọng như:

- nhận message init để đồng bộ trạng thái ban đầu,
- nhận message presence để hiển thị danh sách người dùng online,
- nhận message operation_applied để cập nhật nội dung sau khi server xử lý,
- nhận message cursor để hiển thị con trỏ và vùng chọn của người dùng khác,
- nhận message role_changed hoặc access_removed để cập nhật quyền truy cập realtime.

Ngoài editor realtime, frontend còn có các thành phần giao diện hỗ trợ cộng tác (chức năng nâng cao) :

- DocumentsPage: dashboard quản lý tài liệu, tạo tài liệu, mở tài liệu, xóa tài liệu.
- ShareDialog: giao diện chia sẻ tài liệu và phân quyền owner/editor/viewer.
- PresenceBar: hiển thị danh sách người dùng đang online trong tài liệu.
- HistoryPanel: hiển thị lịch sử chỉnh sửa và hỗ trợ rollback/versioning.
- Các nút Undo/Redo gọi API server để xử lý undo/redo cộng tác.

Như vậy, frontend không chỉ là lớp giao diện mà còn là một client node tham gia vào quá trình truyền thông điệp, đồng bộ trạng thái và xử lý lỗi kết nối trong hệ thống phân tán.

1.2.2 Backend

Backend nằm trong backend/app, sử dụng FastAPI.

Backend đóng vai trò:

- Xác thực và phân quyền: Thực hiện xác thực người dùng bằng cơ chế JWT kết hợp băm bcrypt. Hệ thống tích hợp một bộ dependency chuyên dụng là `get_current_user()` vào mọi endpoint yêu cầu bảo mật nhằm tự động giải mã JWT, tra cứu thực thể người dùng dưới cơ sở dữ liệu và chủ động trả về mã lỗi 401 Unauthorized nếu mã thông báo không hợp lệ. Riêng đối với kết nối WebSocket do đặc thù không hỗ trợ custom header, backend xử lý xác thực bằng cách yêu cầu client truyền token trực tiếp qua query parameter dưới dạng `?token=` và điều hướng qua hàm giải mã riêng biệt `decode_access_token()`
- Quản lý nghiệp vụ tài liệu và quyền hạn: Tiếp nhận và xử lý các yêu cầu CRUD đối với tài liệu cũng như phân định quyền chia sẻ tài liệu giữa các tài khoản người dùng

Quản lý phân quyền và Xác thực kết nối (Permission & Authentication Management)

- Xác thực tầng Handshake: Vì giao thức WebSocket của trình duyệt không cho phép tùy biến HTTP Headers (như Authorization: Bearer <token>), hệ thống chuyển cấu hình xác thực sang dạng sử dụng Query Parameter (`ws://.../ws/documents/{doc_id}?token=<jwt_token>`). Hàm xử lý tại router sẽ bóc tách token này, giải mã thông qua thư viện `jose/jwt` tại lớp `security.py`.
- Kiểm tra quyền truy cập (Permission Checking): Ngay trong quá trình thiết lập kết nối (Handshake), Backend gọi dịch vụ `permissions.py` để tra cứu quyền của người dùng đối với tài liệu yêu cầu trong PostgreSQL/Redis. Chỉ khi người dùng có quyền hợp lệ (owner, editor, viewer), lệnh `websocket.accept()` mới được kích hoạt để mở cổng giao tiếp. Nếu không hợp lệ hoặc token hết hạn, kết nối sẽ bị từ chối lập tức với mã lỗi 4003 Forbidden.

Thiết lập và Mở kết nối WebSocket (Connection Management)

- Hệ thống sử dụng một lớp quản lý tập trung là `ConnectionManager` (nằm trong mạch xử lý của router `documents`). Lớp này duy trì một cấu trúc dữ liệu dạng Dictionary lưu trong bộ nhớ RAM của tiến trình backend: `active_connections: Dict[str, List[WebSocket]]`, trong đó key là `document_id` và value là danh sách các kết nối WebSocket đang hoạt động trên tài liệu đó.
- Khi kết nối được chấp nhận, thực thể WebSocket của client sẽ được đăng ký vào danh sách tương ứng của `document_id` đó để sẵn sàng cho các tác vụ truyền thông điệp nội bộ.

Tiếp nhận Operation từ Client (Receiving Operations)

- Sau khi thiết lập kênh kết nối song công (Duplex), Backend Gateway duy trì một vòng lặp bất đồng bộ `while True` để liên tục lắng nghe các gói tin (frames) gửi lên từ client dưới dạng văn bản định dạng JSON.
- Các gói tin chỉnh sửa văn bản (Operation) do Client gửi lên bắt buộc phải tuân thủ cấu trúc Schema quy định, bao gồm các thông tin: `client_op_id` (mã định danh duy nhất của thao tác), `base_version` (phiên bản tài liệu tại client trước khi gõ), và `operation_delta` (cấu trúc Quill Delta mô tả hành động chèn/xóa/định dạng văn bản).

Điều phối và Gửi Operation vào Message Broker RabbitMQ (Message Queue Ingestion)

- Xử lý phi tập trung (Non-blocking): Để tránh hiện tượng nghẽn mạng khi có hàng trăm người cùng gõ chữ một lúc, Backend Gateway tuyệt đối không giữ kết nối WebSocket để đợi ghi vào DB. Ngay khi nhận được gói tin mã JSON hợp lệ từ client, Backend sử dụng thư viện `aio-pika` để đẩy (Publish) gói tin này vào Exchange của RabbitMQ một cách bất đồng bộ thông qua hàng đợi định sẵn (`doc_operations_queue`).
- Cơ chế này giúp giải phóng tài nguyên của WebSocket Gateway ngay lập tức để tiếp tục nhận keystroke tiếp theo của các client khác, chuyển giao toàn bộ trách nhiệm phân giải xung đột (Operational Transformation) cho cụm OT Worker xử lý ở phía sau.

Điều phối và Gửi Operation vào Message Broker RabbitMQ (Message Queue Ingestion)

- Xử lý phi tập trung (Non-blocking): Để tránh hiện tượng nghẽn mạng khi có hàng trăm người cùng gõ chữ một lúc, Backend Gateway tuyệt đối không giữ kết nối WebSocket để đợi ghi vào DB. Ngay khi nhận được gói tin mã JSON hợp lệ từ client, Backend sử dụng thư viện `aio-pika` để đẩy (Publish) gói tin này vào Exchange của RabbitMQ một cách bất đồng bộ thông qua hàng đợi định sẵn (`doc_operations_queue`).
- Cơ chế này giúp giải phóng tài nguyên của WebSocket Gateway ngay lập tức để tiếp tục nhận keystroke tiếp theo của các client khác, chuyển giao toàn bộ trách nhiệm phân giải xung đột (Operational Transformation) cho cụm OT Worker xử lý ở phía sau.

Tiếp nhận phản hồi và Broadcast sự kiện tới các Client (Event Broadcasting)

- Song song với luồng nhận, Backend Gateway chạy một tiến trình nền (Background Task) lắng nghe hàng đợi phản hồi (`doc_broadcast_queue`) từ

RabbitMQ – nơi mà OT Worker trả về các operation đã được biến đổi và gán số phiên bản chính thức từ server (server_version).

- Khi nhận được thông điệp đã xử lý xong từ Worker, Backend xác định document_id của thông điệp, truy cập vào ConnectionManager để lấy ra danh sách tất cả các kết nối WebSocket đang mở trên tài liệu đó.
- Hệ thống thực hiện một vòng lặp duyệt nhanh và gọi hàm websocket.send_json() để Broadcast (phát sóng đồng thời) gói tin phản hồi tới tất cả các client đang cộng tác trong phòng. Gói tin broadcast này bao gồm: nội dung chữ đã đồng bộ, vị trí con trỏ cập nhật (cursor), thông tin người vừa gõ (presence) và số hiệu phiên bản mới nhất, giúp toàn bộ các trình duyệt hội tụ về cùng một trạng thái hiển thị duy nhất theo thời gian thực.

1.2.3 PostgreSQL

Hệ thống sử dụng cơ sở dữ liệu PostgreSQL làm nơi lưu trữ dữ liệu bền vững duy nhất và tối cao của toàn bộ kiến trúc phân tán. Lớp lưu trữ này có nhiệm vụ đảm bảo an toàn tuyệt đối cho dữ liệu, tránh hiện tượng mất mát thông tin trong các kịch bản bất khả kháng khi bộ nhớ đệm Redis cache bị xóa sạch hoặc tiến trình OT Worker gặp sự cố gián đoạn đột ngột

Để tối ưu hóa quá trình vận hành, toàn bộ schema lưu trữ được định nghĩa song song bằng hai phương thức:

- SQLAlchemy ORM models trong mã nguồn Python (entities.py) phục vụ trực tiếp cho các thao tác truy vấn, cập nhật dữ liệu ở thời gian chạy (runtime) của ứng dụng.
- File SQL DDL thuần (backend/sql/init/001_schema.sql) tích hợp sẵn các cấu trúc trigger, ràng buộc phức tạp để phục vụ cho việc tự động khởi tạo cấu trúc bảng bên trong các container Docker khi triển khai hệ thống.

1. Bảng users (Quản lý thông tin tài khoản)

Tên cột	Kiểu dữ liệu	Mô tả ràng buộc & Chi mục	Ghi chú thiết kế
id	VARCHAR(36)	PRIMARY KEY	Sử dụng định danh duy nhất UUID tự động sinh thông qua hàm gen_random_uuid().

email	VARCHAR(255)	UNIQUE, INDEX	Chuỗi email định danh dùng để đăng nhập hệ thống, được tạo chỉ mục tăng tốc độ tra cứu.
username	VARCHAR(80)	Không ràng buộc duy nhất	Tên hiển thị của người dùng trên giao diện ứng dụng.
password_hash	VARCHAR(255)	Chuỗi mã hóa	Lưu trữ chuỗi mã hóa mật khẩu được xử lý bằng thuật toán bcrypt, không lưu plain-text.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Ghi nhận thời điểm tài khoản người dùng được khởi tạo.

2. Bảng documents (Lưu trữ thông tin tài liệu)

Tên cột	Kiểu dữ liệu	Mô tả ràng buộc & Chỉ mục	Ghi chú thiết kế
id	VARCHAR(36)	PRIMARY KEY	Định danh duy nhất UUID của tài liệu.
title	VARCHAR(255)	DEFAULT 'Untitled document'	Tiêu đề của tài liệu.
content_delta	JSONB	Định dạng cấu trúc phức hợp	Nội dung tài liệu lưu trữ dưới định dạng Quill Delta theo cấu trúc {ops:[...]}. Lưu ý: Cần có chỉ mục GIN để tối ưu truy vấn tìm kiếm nội dung.
content_text	TEXT	Văn bản thuần	Dữ liệu plain-text trích xuất từ delta dùng cho tính năng xem trước (preview) và tìm kiếm nhanh.
owner_id	VARCHAR(36)	FOREIGN KEY → users.id (ON DELETE CASCADE)	Định danh người sở hữu tối cao.
version	INTEGER	Số nguyên tăng dần	Phiên bản hiện tại của tài liệu, tự động cộng 1 sau mỗi operation.

vector_clock	JSONB	Đồng hồ Vector	Lưu vết trạng thái phân tán dưới dạng map {"server": N, "user_id": M, ...}.
created_at	TIMESTAMP	Thời gian thực	Ghi nhận thời điểm tài liệu được tạo ra.
updated_at	TIMESTAMP	Tự động cập nhật	Tích hợp Trigger set_updated_at() tự động cập nhật khi có thay đổi.

3. Bảng document_permissions (Quản lý phân quyền tài liệu)

Tên cột	Kiểu dữ liệu	Mô tả ràng buộc & Chỉ mục	Ghi chú thiết kế
id	VARCHAR(36)	PRIMARY KEY	Định danh UUID của bản ghi phân quyền.
document_id	VARCHAR(36)	FOREIGN KEY → documents.id (ON DELETE CASCADE)	Liên kết mã tài liệu được chia sẻ.
user_id	VARCHAR(36)	FOREIGN KEY → users.id (ON DELETE CASCADE)	Liên kết mã người dùng được nhận quyền.
role	VARCHAR(20)	CHECK ('owner', 'editor', 'viewer')	Vai trò của người dùng trên tài liệu.
created_at	TIMESTAMP	Thời gian thực	Ghi nhận thời điểm chia sẻ quyền thành công.

4. Bảng document_operations (Nhật ký lịch sử thao tác chỉnh sửa)

Tên cột	Kiểu dữ liệu	Mô tả ràng buộc & Chỉ mục	Ghi chú thiết kế
id	VARCHAR(36)	PRIMARY KEY	Định danh UUID của thao tác chỉnh sửa.

document_id	VARCHAR(36)	FOREIGN KEY → documents.id (ON DELETE CASCADE)	Liên kết mã tài liệu.
user_id	VARCHAR(36)	FOREIGN KEY → users.id (ON DELETE SET NULL)	Người thực hiện thao tác.
client_op_id	VARCHAR(120)	INDEX	Idempotency key - chống trùng lặp thao tác do lỗi mạng.
operation_delta	JSONB	Quill Delta gốc	Thao tác chỉnh sửa nguyên bản từ client.
transformed_delta	JSONB	Quill Delta sau khi transform	Thao tác sau khi OT Worker xử lý biến đổi.
vector_clock	JSONB	GIN Index	Trạng thái đồng hồ vector tại thời điểm áp dụng.
base_version	INTEGER	Số nguyên	Phiên bản tài liệu mà client đang dựa vào.
server_version	INTEGER	Số nguyên	Phiên bản chính thức do server gán sau khi áp dụng.
causal_relation	VARCHAR(30)	Giá trị: before, after, concurrent, equal	Quan hệ nhân quả của thao tác.
created_at	TIMESTAMP	Thời gian thực	Thời điểm operation được lưu và áp dụng.

5. Bảng document version:

Tên cột	Kiểu dữ liệu	Mô tả ràng buộc & Chỉ mục	Ghi chú thiết kế
id	VARCHAR(36)	PRIMARY KEY	Định danh duy nhất UUID của bản ghi version.

document_id	VARCHAR(36)	FOREIGN KEY → documents.id (ON DELETE CASCADE)	Liên kết với tài liệu. Xóa tài liệu thì toàn bộ lịch sử version bị xóa theo.
version	INTEGER	UNIQUE cùng document_id	Số thứ tự phiên bản, bắt đầu từ 0 khi tạo tài liệu, tăng dần sau mỗi thay đổi.
content_delta	JSONB	Định dạng cấu trúc phức hợp	Snapshot đầy đủ nội dung tài liệu tại thời điểm version này, lưu dạng Quill Delta {ops:[...]}. Không lưu diff để đơn giản hóa rollback.
content_text	TEXT	Văn bản thuần	Plain text trích xuất từ delta, dùng để hiển thị preview trong HistoryPanel.
vector_clock	JSONB	Định dạng cấu trúc phức hợp	Trạng thái đồng hồ vector tại thời điểm tạo version, dùng xác định quan hệ nhân quả.
created_by	VARCHAR(36)	FOREIGN KEY → users.id (ON DELETE SET NULL)	Định danh người thực hiện thay đổi tạo ra version này.
operation_id	VARCHAR(36)	FOREIGN KEY → document_operations.id	Liên kết với operation cụ thể đã tạo ra version này.
action	VARCHAR(30)	CHECK IN ('create', 'edit', 'rollback', 'undo', 'redo')	Loại hành động tạo ra version. Dùng để phân biệt khi thực hiện undo/redo tránh loop.
source_version	INTEGER	Số nguyên	Số version trước khi thực hiện action. Ví dụ đang ở version 10 rồi undo thì source_version = 10.
target_version	INTEGER	Số nguyên	Version đích muốn quay về. Dùng cho rollback/undo/redo để biết anchor mà không cần duyệt lại toàn bộ lịch sử.

created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm tạo version.
------------	-----------	------------------------------	------------------------

1.2.4 Redis

Redis được sử dụng làm bộ nhớ cache tốc độ cao.

Redis lưu:

- snapshot mới nhất của tài liệu,
- version hiện tại,
- vector clock hiện tại,
- trạng thái tạm thời của document.

Redis lưu snapshot mới nhất của tài liệu, version và vector_clock hiện tại để tăng tốc đọc/ghi trạng thái document. Các trạng thái realtime như presence và cursor được quản lý tạm thời trong ConnectionManager của backend, không lưu bền vững vào Redis/PostgreSQL.

1.2.5 RabbitMQ

RabbitMQ đóng vai trò message broker trung gian giữa Gateway và Worker.

Nhiệm vụ:

- nhận operation từ backend,
- đưa operation vào queue,
- đảm bảo xử lý tuần tự,
- giảm coupling giữa các thành phần.

Nhờ RabbitMQ:

- backend không bị block khi có nhiều thao tác realtime,
- hệ thống dễ scale worker theo chiều ngang,
- tránh ghi đồng thời trực tiếp vào tài liệu.

1.2.6 OT Worker

OT Worker nằm trong worker/main.py.

Worker chịu trách nhiệm:

- nhận operation từ RabbitMQ,
- xử lý Operational Transformation
- kiểm tra version,
- kiểm tra vector clock,
- transform operation khi có xung đột,
- cập nhật snapshot,

- lưu operation history,
- publish event trở lại backend.

Worker chính là thành phần quyết định trạng thái cuối cùng của tài liệu.

1.3 Luồng Chính Sửa Realtime

Luồng chỉnh sửa realtime:

1. Người dùng chỉnh sửa nội dung trong ReactQuill.
2. Hàm onChange() của frontend được kích hoạt.
3. Frontend kiểm tra thao tác có phải do user tạo ra hay không và user có quyền sửa hay không.
4. Frontend tạo operation_delta từ Quill Delta, gắn kèm base_version, vector_clock, client_id và client_op_id.
5. Operation được đưa vào pending queue để tránh mất dữ liệu nếu mạng lỗi.
6. Nếu WebSocket đang kết nối, hàm sendQueuedOperation() gửi operation tới FastAPI Gateway.
7. Backend Gateway nhận operation, kiểm tra quyền truy cập, sau đó publish operation vào RabbitMQ.
8. OT Worker nhận operation từ RabbitMQ, kiểm tra version/vector clock và transform operation nếu có xung đột.
9. Worker cập nhật snapshot tài liệu, tăng version, lưu dữ liệu vào Redis/PostgreSQL.
10. Worker publish event operation_applied.
11. Backend nhận event và broadcast operation_applied tới các client đang mở tài liệu.
12. Frontend nhận operation_applied, cập nhật version/vector_clock và gọi applyServerDocument() nếu cần.
13. Các client khác nhìn thấy nội dung thay đổi gần realtime.

Luồng mở tài liệu

1. User chọn một tài liệu từ dashboard.
2. Frontend gọi GET /api/documents/{document_id} để lấy title, content_delta, version, vector_clock và role.
3. Frontend mở WebSocket bằng connectSocket().

4. Backend xác thực JWT token và kiểm tra quyền truy cập tài liệu.
5. Nếu hợp lệ, backend chấp nhận WebSocket.
6. Server gửi message init gồm nội dung hiện tại, version, vector_clock, role và danh sách user online.
7. Frontend gọi applyServerDocument() để render nội dung vào ReactQuill.
8. Frontend enable hoặc disable editor dựa trên role.
9. Server gọi broadcast_presence() để cập nhật danh sách user online cho các client khác.

Luồng này áp dụng Client-Server, Access Control, State Synchronization và Group Membership.

1.4 Luồng phân quyền

Hệ thống soạn thảo cộng tác đòi hỏi một cơ chế kiểm soát truy cập cực kỳ nghiêm ngặt nhằm đảm bảo tính toàn vẹn của tài liệu và an toàn thông tin giữa các tầng cộng tác viên. Thành viên 2 đã thiết kế và triển khai trọn vẹn mô hình phân quyền bảo mật tầng sâu tại module backend/app/services/permissions.py, kết hợp chặt chẽ giữa các HTTP REST API và kênh truyền dữ liệu Realtime WebSocket.

1.4.1 Kiến trúc phân quyền 3 tầng (3-Tier Permission Architecture)

Thay vì kiểm tra quyền một cách thủ công tại từng endpoint, hệ thống sử dụng cơ chế Decorator/Dependency Injection của FastAPI để xây dựng 3 tầng rào chắn bảo mật (Guards). Mọi request từ client bắt buộc phải đi qua các tầng kiểm tra này trước khi chạm tới lớp xử lý nghiệp vụ:

1. Tầng require_document_access (Quyền truy cập cơ bản - Đọc/Xem)
 - Phạm vi áp dụng: Các endpoint phục vụ việc đọc dữ liệu và kiểm tra lịch sử, bao gồm: GET chi tiết nội dung tài liệu, GET /history (xem nhật ký thao tác), và GET /versions (xem danh sách các bản đóng băng snapshot).
 - Luật xử lý: Hệ thống tra cứu bảng document_permissions và bảng documents. Nếu người dùng có vai trò là owner, editor hoặc viewer, request sẽ được phê duyệt thông qua. Nếu không có bản ghi hợp lệ, hệ thống lập tức từ chối và trả về mã lỗi 403 Forbidden.
2. Tầng require_document_edit (Quyền thay đổi trạng thái tài liệu)
 - Phạm vi áp dụng: Các endpoint làm thay đổi cấu trúc cấu hình hoặc dòng thời gian của tài liệu nhưng không phá hủy tài liệu, bao gồm: PATCH (đổi tiêu đề tài liệu), POST /rollback (khôi phục về phiên bản lịch sử), cùng các logic xử lý undo và redo tập trung tại server.

- Luật xử lý: Rào chắn này chỉ chấp nhận người dùng có vai trò mã hóa là owner hoặc editor. Người dùng có quyền viewer cố tình gửi request tới các endpoint này sẽ bị chặn đứng ngay tại tầng trung gian.
3. Tầng require_owner (Quyền chủ sở hữu tối cao)
- Phạm vi áp dụng: Các endpoints có tính chất quản trị, ảnh hưởng trực tiếp đến quyền sinh tồn của tài liệu hoặc quyền truy cập của người khác, bao gồm: DELETE tài liệu, POST /share (cấp quyền/chia sẻ cho người dùng mới), và DELETE /permissions (thu hồi quyền của một thành viên).
 - Luật xử lý: Tầng bảo mật nghiêm ngặt nhất. Hệ thống thực hiện so khớp trực tiếp mã user_id trích xuất từ JWT với trường owner_id trong bảng documents. Chỉ duy nhất chủ sở hữu tài liệu được phép thực thi.

1.4.2 Cơ chế thu hồi quyền thời gian thực và Ngắt kết nối lập tức (Realtime Revocation & WebSocket Kick)

- Vấn đề kỹ thuật của hệ thống thông thường: Trong các kiến trúc truyền thống, khi Owner thực hiện thu hồi quyền của một Editor, Editor đó vẫn có thể tiếp tục gõ chữ và gửi lệnh chỉnh sửa qua kết nối WebSocket đã thiết lập trước đó, do kết nối WebSocket không tự động kiểm tra lại quyền trên mỗi frame dữ liệu truyền tải. Quyền hạn chỉ thực sự bị chặn khi người dùng đó tải lại (reload) trang hoặc mã mã thông báo (token) hết hạn.
- Giải pháp xử lý Realtime của Thành viên 2: Để giải quyết triệt để lỗ hổng bảo mật này, hệ thống đã triển khai luồng phối hợp bất đồng bộ liên tầng
- Chi tiết các bước vận hành:
 - a. Giai đoạn kích hoạt: Khi Owner nhấn nút xóa quyền của một thành viên trên giao diện ShareDialog.jsx, Frontend gửi một request DELETE tới endpoint /documents/{id}/permissions.
 - b. Xử lý tại Backend: Backend Gateway tiếp nhận request, tiến hành xóa bản ghi tương ứng trong bảng document_permissions tại PostgreSQL, đồng thời xóa khóa bộ nhớ đệm của user đó trên Redis cache để đồng bộ dữ liệu.
 - c. Kích hoạt Broker: Ngay sau khi dòng dữ liệu trong DB được cập nhật thành công, Backend tạo ra một thông điệp sự kiện thu hồi quyền (Revocation Event) chứa các metadata: { "event": "permission_revoked", "document_id": doc_id, "user_id": target_user_id } và phát (publish) thẳng vào Exchange hệ thống của RabbitMQ.
 - d. Xử lý tại WebSocket Gateway: Kênh Realtime WebSocket Gateway liên tục lắng nghe hàng đợi này. Khi nhận được thông điệp từ RabbitMQ, Gateway sẽ quét cấu trúc quản lý kết nối ConnectionManager. Hệ thống tìm chính xác thực thể kết nối WebSocket đang mở của target_user_id nằm trong phân vùng của document_id.

- e. Thực thi lệnh Kick: WebSocket Gateway chủ động đóng kết nối mạng của thành viên bị thu hồi quyền bằng phương thức `websocket.close(code=4003)`.
- f. Phản hồi tại Frontend: Phía Client của người dùng bị thu hồi quyền nhận được tín hiệu ngắt kết nối từ server với mã trạng thái đặc biệt 4003. Lớp mã nguồn Frontend (do Thành viên 2 cấu hình) sẽ bắt lấy mã lỗi này, lập tức hiển thị một thông báo cảnh báo (Toast thông báo quyền truy cập đã thay đổi) và tự động kích hoạt điều hướng (Kéo người dùng ra khỏi giao diện soạn thảo văn bản, đẩy về trang chủ `DocumentsPage.jsx`).

Toàn bộ quá trình thu hồi quyền và đẩy người dùng ra khỏi tài liệu diễn ra hoàn toàn tự động theo thời gian thực dưới 500ms mà không yêu cầu người dùng bị thu hồi quyền phải thực hiện bất kỳ thao tác tải lại trang (No page reload required) nào, đảm bảo tính an toàn tuyệt đối cho tài nguyên thông tin.

1.5 Docker Deployment

Hệ thống được triển khai bằng Docker Compose, trong đó mỗi thành phần chạy trong một container riêng để mô phỏng kiến trúc phân tán nhiều tầng.

Các service chính gồm:

- frontend: chạy giao diện React + ReactQuill.
- backend: chạy FastAPI Gateway, REST API và WebSocket.
- postgres: lưu dữ liệu bền vững như user, document, permission, version, operation history.
- redis: cache snapshot tài liệu, version và vector_clock.
- rabbitmq: hàng đợi trung gian để chuyển operation từ backend sang worker.
- worker: xử lý OT, cập nhật tài liệu và publish event sau khi xử lý xong.

Luồng hoạt động khi chạy Docker:

1. PostgreSQL, Redis và RabbitMQ khởi động làm hạ tầng.
2. Backend kết nối tới database, Redis và RabbitMQ.
3. Worker lắng nghe operation từ RabbitMQ.
4. Frontend gửi request REST API/WebSocket tới backend.
5. Khi user chỉnh sửa, operation đi theo luồng: Frontend → Backend → RabbitMQ → Worker → Backend broadcast → Clients.

Việc dùng Docker giúp các thành phần được tách biệt, dễ chạy lại môi trường, dễ triển khai thử nghiệm và phù hợp với mô hình hệ thống phân tán vì các service giao tiếp với nhau qua network nội bộ thay vì chạy chung trong một tiến trình.

MÔ HÌNH ĐỒNG BỘ DỮ LIỆU

2.1 Mô hình đồng bộ dữ liệu

Hệ thống sử dụng mô hình:

Server-Authoritative Realtime Synchronization

Ở phía frontend, mô hình Server-Authoritative được thể hiện qua hàm `applyServerDocument()`. Client có thể hiển thị tạm các thao tác local để tăng trải nghiệm realtime, nhưng trạng thái cuối cùng của tài liệu vẫn được xác nhận bởi server. Khi nhận snapshot hoặc event `operation_applied` từ server, frontend cập nhật lại `content_delta`, `version` và `vector_clock` theo dữ liệu server gửi về.

Để tránh mất thao tác đang gõ, frontend không ghi đè editor một cách trực tiếp. Thay vào đó, frontend set nội dung theo snapshot server trước, sau đó apply lại các operation local còn pending. Cách xử lý này giúp client vẫn giữ được thao tác chưa đồng bộ nhưng vẫn hội tụ về trạng thái chuẩn của server.

Trong mô hình này:

- server là nguồn dữ liệu chính,
- client không tự xác nhận trạng thái cuối cùng,
- mọi operation phải được worker xử lý trước khi broadcast.

Cách tiếp cận này giúp:

- đảm bảo tính nhất quán,
- giảm lỗi ghi đè,
- kiểm soát xung đột tập trung.

2.2 Dữ liệu được đồng bộ

Trong một kiến trúc soạn thảo cộng tác phân tán, hệ thống không chỉ đơn thuần đồng bộ hóa chuỗi ký tự văn bản thô, mà còn phải tiến hành truyền tải và đồng bộ một tập hợp phức hợp các siêu dữ liệu (metadata) đi kèm. Việc này đóng vai trò quyết định trong việc bảo toàn quan hệ nhân quả, hóa giải xung đột đồng thời và duy trì tính nhất quán tối cao của tài liệu trên toàn mạng lưới.

Để tối ưu hóa hiệu năng, các thành phần dữ liệu đồng bộ được phân định rõ ràng thành hai nhóm bản chất: Trạng thái cứng (Hard State) đòi hỏi lưu trữ bền vững

lâu dài và Trạng thái mềm (Soft State) chỉ tồn tại tạm thời trong runtime để phục vụ trải nghiệm UI/UX trực quan.

2.2.1 Bảng đặc tả các trường dữ liệu (Schema thông điệp).

Trường	Ý nghĩa
content_delta	Nội dung tài liệu dạng Quill Delta
version	Phiên bản hiện tại
vector_clock	Đồng hồ vector
operation_delta	Operation từ client
transformed_delta	Operation sau transform
server_version	Version do server xác nhận
client_op_id	Mã định danh operation do client tạo, dùng để nhận biết operation đã được server xác nhận và tránh xử lý trùng khi retry
cursor.index	Vị trí con trỏ của user trong tài liệu
cursor.length	Độ dài vùng chọn, nếu length > 0 nghĩa là user đang bôi chọn văn bản
presence.users	Danh sách người dùng đang online trong tài liệu
role	Quyền hiện tại của user trong document: owner/editor/viewer
offline queue	Danh sách operation local chưa được server xác nhận

2.2.2 Triết lý phân tách Hard State và Soft State trong đồng bộ

Để hệ thống hoạt động ổn định với độ trễ tối thiểu, kiến trúc phân tán này thực hiện phân tách triệt để phương thức truyền thông và lưu trữ dữ liệu tùy theo tính chất của chúng

Cơ chế quản lý Trạng thái cứng (Hard State): Bao gồm content_delta, version, vector_clock, và lịch sử operation_delta. Đây là các thông tin cốt lõi cấu thành nên nội dung lịch sử văn bản, bắt buộc phải được đồng bộ hóa nghiêm ngặt, xử lý tuần tự qua hàng đợi RabbitMQ, ghi nhận vào bộ nhớ đệm Redis cache để tăng tốc đọc/ghi và lưu trữ bền vững xuống cơ sở dữ liệu PostgreSQL nhằm đảm bảo an toàn, chống mất mát dữ liệu khi có sự cố sập nguồn phần cứng.

Cơ chế quản lý Trạng thái mềm (Soft State): Bao gồm dữ liệu về cursor (con trỏ chuột) và presence (trạng thái online). Do đặc thù các thao tác di chuyển chuột, bôi đen văn bản diễn ra liên tục với tần suất cực cao (vài mươi mili-giây một lần), hệ thống tuyệt đối không lưu các trường này vào Redis hay PostgreSQL nhằm giải phóng hoàn toàn áp dụng I/O cho cơ sở dữ liệu. Thay vào đó, dữ liệu được Backend Gateway lưu tạm thời trên bộ nhớ RAM thông qua cấu trúc điều phối ConnectionManager và được phát sóng trực tiếp (Realtime Broadcast) bằng WebSocket tới các client cùng phòng. Khi người dùng ngắt kết nối (disconnect), toàn bộ soft state này của họ sẽ lập tức bị hủy bỏ và xóa sạch khỏi bộ nhớ hệ thống.

2.2.3 Chi tiết các thành phần dữ liệu đồng bộ cốt lõi

Để đảm bảo tính nhất quán tuyến tính, bảo toàn quan hệ nhân quả và tối ưu hóa trải nghiệm cộng tác, hệ thống tiến hành phân rã và đồng bộ hóa 6 thành phần dữ liệu cốt lõi sau:

1. Nội dung tài liệu (Document Content)

- Bản chất: Trạng thái cứng (Hard State) – Yêu cầu bảo toàn tuyệt đối.
- Định dạng cấu trúc: Chuỗi cấu trúc dữ liệu phức hợp dạng Quill Delta (JSONB) dưới dạng tập hợp mảng thao tác biểu diễn hành động văn bản (ví dụ: {"ops": [{"insert": "Hello "}, {"insert": "World", "attributes": {"bold": true}}]}).
- Cơ chế đồng bộ: Đây là trạng thái đích (converged state) mà tất cả các client bắt buộc phải hội tụ về. Nội dung tài liệu được lưu đồng thời tại RAM của Client (trình soạn thảo ReactQuill), lưu đệm tại bộ nhớ Redis cache (để phục vụ các request đọc nhanh GET /documents/{id}) và lưu trữ bền vững tại trường content_delta của bảng documents trong PostgreSQL. Khi có thay đổi, nội dung

này chỉ được cập nhật sau khi thuật toán OT xử lý phân giải xung đột thành công.

2. Số hiệu phiên bản (Version)

- Bản chất: Trạng thái cứng (Hard State).
- Định dạng cấu trúc: Số nguyên tăng đơn điệu (INTEGER), bắt đầu từ 0 khi khởi tạo tài liệu.
- Cơ chế đồng bộ: Số hiệu phiên bản do máy chủ toàn quyền quản lý và là căn cứ tối cao để xác định tính đồng thời.
 - Phía Client: Mỗi client giữ một biến `base_version` (phiên bản mà client đang nhìn thấy khi gõ chữ).
 - Phía Server: Khi một operation được áp dụng thành công vào cơ sở dữ liệu, số hiệu version của tài liệu tăng lên 1. Chỉ số phiên bản này được đính kèm vào gói tin broadcast để ép tất cả các client nâng cấp trạng thái cục bộ, đảm bảo tính đồng bộ dòng thời gian.

3. Đồng hồ Vector (Vector Clock)

- Bản chất: Trạng thái cứng (Hard State).
- Định dạng cấu trúc: Cấu trúc cấu hình dạng Map/Dictionary (JSONB) lưu vết trạng thái của các nút phân tán dưới dạng: `{"server": 14, "client_a": 8, "client_b": 6}`.
- Cơ chế đồng bộ: Đồng hồ Vector dùng để xác định quan hệ nhân quả (Causal Consistency) giữa các thao tác chỉnh sửa trong môi trường bất đồng bộ. Trước khi gửi một operation lên mạng, client tăng chỉ số đồng hồ của chính mình lên 1 và đính kèm vector clock này vào gói tin. Server dựa vào vector clock nhận được để so sánh với trạng thái hiện tại nhằm phát hiện xem thao tác này thuộc quan hệ: xảy ra trước (before), xảy ra sau (after), hay đồng thời (concurrent), từ đó quyết định có cần kích hoạt hàm biến đổi của thuật toán OT hay không.

4. Nhật ký thao tác (Operation Log)

- Bản chất: Trạng thái cứng (Hard State).

- Định dạng cấu trúc: Cấu trúc đối tượng JSON lưu trữ trong bảng `document_operations`.
- Cơ chế đồng bộ: Hệ thống không truyền đi toàn bộ file văn bản khi có người gõ chữ, mà chỉ truyền đi các vi-thao tác (atomic operations). Một bản ghi Op Log đồng bộ bao gồm: `client_op_id` (khóa chống trùng lặp), `operation_delta` (nội dung gõ nguyên bản), `transformed_delta` (nội dung sau khi OT phân giải xung đột) và `server_version`. Nhật ký này được đồng bộ bất đồng bộ qua hàng đợi RabbitMQ, sau đó lưu vĩnh viễn vào PostgreSQL để làm cơ sở cho bộ xử lý OT tính toán khoảng cách phiên bản và phục vụ tính năng Collaborative Undo/Redo tầng Server.

5. Trạng thái con trỏ (Cursor Status)

- Bản chất: Trạng thái mềm (Soft State) – Tần suất thay đổi cực cao.
- Định dạng cấu trúc: Đối tượng JSON mô tả tọa độ tuyến tính bao gồm hai trường: `index` (vị trí con trỏ trong văn bản) và `length` (độ dài vùng văn bản đang bôi chọn). Ví dụ: `{"index": 12, "length": 0}`.
- Cơ chế đồng bộ: Để tối ưu hiệu năng và tránh gây nghẽn I/O hệ thống, trạng thái con trỏ tuyệt đối không được ghi vào cơ sở dữ liệu (PostgreSQL hay Redis). Khi người dùng di chuyển chuột hoặc bôi đen văn bản, Frontend phát gói tin cursor qua kết nối WebSocket. Backend Gateway nhận được sẽ dùng ConnectionManager duyệt nhanh danh sách phòng và thực hiện Broadcast trực tiếp tới các client khác cùng phòng để hiển thị thanh con trỏ màu (caret) tương ứng theo thời gian thực. Trạng thái này tự động mất đi khi người dùng đóng trình duyệt.

6. Trạng thái hiện diện người dùng (User Presence)

- Bản chất: Trạng thái mềm (Soft State).
- Định dạng cấu trúc: Mảng danh sách các đối tượng (Array of Objects) chứa thông tin người dùng: `[{"user_id": "uuid-1", "username": "Hai", "color": "#ff5733", "role": "editor"}]`.

- Cơ chế đồng bộ: Thành phần này quản lý danh sách những ai đang online trong cùng một tài liệu. Cơ chế đồng bộ dựa hoàn toàn vào vòng đời của kết nối WebSocket:
 - Khi một kết nối WebSocket được thiết lập thành công (`websocket.accept()`), Backend thêm user vào danh sách presence của tài liệu đó và phát sự kiện `"user_joined"` tới cả phòng.
 - Khi kết nối bị ngắt (`websocket.close()` hoặc mất mạng), hệ thống tự động xóa user ra khỏi bộ nhớ RAM của gateway và phát sự kiện `"user_left"`. Frontend nhận các sự kiện này để cập nhật danh sách avatar góc phải màn hình trực quan mà không cần lưu vết xuống DB.

2.3 Cơ chế chống trùng operation

Do client có thể reconnect và gửi lại operation pending, mỗi operation được gắn `client_op_id`. Worker kiểm tra `client_op_id` trước khi xử lý. Nếu operation đã tồn tại trong lịch sử, Worker không áp dụng lại vào tài liệu mà trả về `duplicate_ack/operation_applied` tương ứng. Cơ chế này giúp tránh lỗi một thao tác bị áp dụng hai lần, ví dụ user gõ "a" nhưng do retry mà thành "aa".

2.4 Luồng Đồng Bộ Khi Người Dùng Chỉnh Sửa

Quy trình đồng bộ khi user chỉnh sửa tài liệu:

1. User chỉnh sửa nội dung trong ReactQuill.
2. Hàm `onChange()` của frontend được kích hoạt.
3. Frontend kiểm tra thao tác có phải do user tạo ra hay không và user có quyền sửa hay không.
4. Frontend tạo `operation_delta` từ Quill Delta.
5. Operation được gắn thêm `document_id`, `user_id`, `base_version`, `vector_clock` và `client_op_id`.
6. Operation được lưu vào pending queue trước khi gửi để tránh mất dữ liệu nếu mạng lỗi.
7. Nếu WebSocket đang mở, `sendQueuedOperation()` gửi operation tới backend.

8. Backend kiểm tra quyền và publish operation vào RabbitMQ.
9. Worker đọc snapshot hiện tại từ Redis/PostgreSQL.
10. Worker kiểm tra version và vector clock.
11. Nếu document đã có thay đổi mới hơn, Worker transform operation bằng OT.
12. Worker áp dụng operation vào snapshot hiện tại.
13. Worker tăng version và cập nhật vector_clock.
14. Worker lưu snapshot mới vào Redis và PostgreSQL.
15. Worker publish event operation_applied.
16. Backend nhận event và broadcast tới các client đang mở tài liệu.
17. Frontend nhận operation_applied.
18. Nếu operation_applied tương ứng với client_op_id trong pending queue, frontend xóa operation đó khỏi queue.
19. Frontend cập nhật version/vector_clock và gọi applyServerDocument() nếu cần.
20. Các client hội tụ về cùng trạng thái tài liệu.

Luồng này kết hợp các cơ chế operation-based synchronization, versioning, vector clock, idempotency và eventual consistency.

2.5 Vai Trò Của RabbitMQ

RabbitMQ giúp:

- tách Gateway khỏi Worker,
- xử lý bất đồng bộ,
- giảm tải backend,
- tránh race condition,
- đảm bảo thứ tự xử lý operation.

Nếu không có RabbitMQ:

- backend phải tự xử lý đồng thời,
- dễ xảy ra ghi đè dữ liệu,
- khó mở rộng hệ thống.

2.6 Vai Trò Của Redis Và PostgreSQL

Redis

Redis dùng để:

- cache snapshot,
- đọc nhanh document,
- giảm truy vấn database,
- tăng tốc xử lý OT.

PostgreSQL

PostgreSQL dùng để:

- lưu dữ liệu lâu dài,
- lưu lịch sử operation,
- phục hồi dữ liệu khi cache mất.

Hai hệ thống phối hợp theo mô hình:

- Redis → tốc độ,
- PostgreSQL → độ bền dữ liệu.

2.7 Xử Lý Xung Đột

Hệ thống dùng Operational Transformation (OT) để xử lý chỉnh sửa đồng thời.

Ví dụ:

- User A và User B cùng sửa ở version 5.
- Operation của A được xử lý trước → version 6.
- Operation của B tới sau:
 - worker thấy base_version = 5,
 - document hiện tại đã là version 6.
- Worker transform operation của B dựa trên operation của A.
- Sau đó mới áp dụng operation của B → version 7.

Nhờ OT:

- các thay đổi không ghi đè trực tiếp,
- tài liệu vẫn hội tụ về cùng trạng thái cuối cùng.

2.8 Mô Hình Nhất Quán

Hệ thống sử dụng:

Eventual Consistency gần realtime

Điều này có nghĩa:

- tại một thời điểm ngắn, các client có thể chưa giống nhau hoàn toàn,
- sau khi worker xử lý và broadcast xong, tất cả client sẽ hội tụ về cùng trạng thái.

Server vẫn là nguồn dữ liệu cuối cùng đáng tin cậy.

2.9 Đồng bộ trạng thái realtime UI

Bên cạnh nội dung tài liệu, hệ thống còn đồng bộ các trạng thái realtime phục vụ trải nghiệm cộng tác như danh sách user online, con trỏ, vùng chọn và quyền truy cập.

Các trạng thái như cursor, selection và presence được xem là soft state. Đây là các trạng thái tạm thời, không cần lưu bền vững vào database. Khi user rời khỏi tài liệu, các trạng thái này có thể bị xóa khỏi bộ nhớ.

Luồng đồng bộ cursor:

1. User di chuyển con trỏ hoặc bôi chọn văn bản.
2. Frontend gọi `onSelectionChange()` để lấy index và length.
3. Client gửi message cursor lên server qua WebSocket.
4. Backend gọi `update_cursor()` để cập nhật cursor tạm thời trong `ConnectionManager`.
5. Server broadcast cursor cho các client khác trong cùng document.
6. Frontend client khác dùng `quill-cursors` để hiển thị con trỏ/vùng chọn.

Luồng đồng bộ presence:

1. User mở tài liệu và kết nối WebSocket.
2. Backend thêm user vào danh sách active của document.
3. Server gọi `broadcast_presence()`.
4. Frontend nhận message presence và cập nhật `PresenceBar`.

5. Khi user disconnect, server xóa user khỏi danh sách active và broadcast lại presence mới.

Cách xử lý này áp dụng các lý thuyết Presence System, Group Membership, Soft State và Realtime Broadcast trong hệ thống phân tán.

2.10 Xử lý offline, reconnect và pending queue

Trong hệ thống phân tán, một client có thể bị mất kết nối trong khi server và các client khác vẫn hoạt động bình thường. Đây là dạng partial failure. Để tránh mất dữ liệu khi mạng lỗi, frontend sử dụng pending queue để lưu tạm các operation chưa được server xác nhận.

Luồng xử lý khi mất kết nối:

1. WebSocket bị ngắt.
2. Frontend chuyển trạng thái giao diện sang Offline.
3. Nếu user tiếp tục chỉnh sửa, hàm onChange() vẫn tạo operation.
4. Operation được lưu vào pending queue/localStorage thay vì bị mất.
5. Hàm scheduleReconnect() tự động thử kết nối lại WebSocket.
6. Khi kết nối phục hồi, connectSocket() mở lại WebSocket.
7. Hàm flushOfflineQueue() gửi lại các operation đang chờ.
8. Server xử lý operation và trả về operation_applied.
9. Frontend nhận xác nhận từ server rồi xóa operation tương ứng khỏi queue.

Cách xử lý này áp dụng các lý thuyết Partial Failure Handling, Fault Tolerance, Retry Mechanism và Eventual Consistency. Nhờ đó, thao tác của người dùng không bị mất khi mạng bị gián đoạn tạm thời.

2.11 Undo/Redo trong môi trường cộng tác

Trong editor thông thường, undo/redo có thể xử lý local trên từng máy. Tuy nhiên, trong hệ thống nhiều người cùng chỉnh sửa, nếu mỗi client tự undo local thì nội dung giữa các client có thể bị lệch nhau. Vì vậy hệ thống xử lý undo/redo theo hướng collaborative undo/redo, tức là thao tác undo/redo được gửi lên server để xử lý trên trạng thái tài liệu chung.

Luồng xử lý undo:

1. User bấm Undo hoặc Ctrl + Z trên giao diện.
2. Frontend kiểm tra user có quyền chỉnh sửa hay không.
3. Hàm collaborativeUndo() gọi REST API undo lên backend.

4. Backend xử lý undo dựa trên DocumentVersion/snapshot đã lưu, tạo một version mới tương ứng với trạng thái sau undo.
5. Backend publish event để các client đang mở tài liệu đồng bộ lại.
6. Frontend nhận content_delta, version và vector_clock mới rồi gọi applyServerDocument().
7. Các client khác nhận event cập nhật và đồng bộ lại nội dung.

Luồng redo tương tự, thông qua hàm collaborativeRedo().

Cách xử lý này giúp undo/redo không còn là thao tác local riêng của từng client, mà trở thành thao tác trên trạng thái tài liệu chung. Điều này giúp đảm bảo consistency giữa các user đang cùng chỉnh sửa.

GIAO THỨC TRAO ĐỔI DỮ LIỆU

Hệ thống sử dụng hai nhóm giao thức chính:

- HTTP REST API cho các thao tác quản lý như đăng ký, đăng nhập, tạo tài liệu, chia sẻ tài liệu.
- WebSocket cho trao đổi dữ liệu realtime khi nhiều người cùng chỉnh sửa tài liệu.

Ngoài ra, bên trong backend còn dùng RabbitMQ message protocol để truyền operation giữa FastAPI Gateway và OT Worker.

3.1 HTTP REST API

HTTP REST được dùng cho các thao tác không yêu cầu realtime.

Các API chính:

- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/me
- GET /api/documents
- POST /api/documents
- GET /api/documents/{document_id}
- PATCH /api/documents/{document_id}
- DELETE /api/documents/{document_id}
- GET /api/documents/{document_id}/permissions

- POST /api/documents/{document_id}/share
- DELETE /api/documents/{document_id}/permissions/{permission_id}
- GET /api/documents/{document_id}/history

Client gửi JWT token trong HTTP header:

- Authorization: Bearer <access_token>

Ví dụ response khi lấy chi tiết tài liệu:

```
{
  "id": "document-id",
  "title": "Untitled document",
  "owner_id": "user-id",
  "role": "owner",
  "version": 3,
  "created_at": "2026-05-25T10:00:00",
  "updated_at": "2026-05-25T10:05:00",
  "content_delta": {
    "ops": [
      { "insert": "Hello world\n" }
    ]
  },
  "vector_clock": {
    "user-1": 2,
    "server": 3
  }
}
```

Frontend sử dụng HTTP REST API cho các thao tác không yêu cầu realtime như đăng nhập, lấy danh sách tài liệu, tạo tài liệu, lấy chi tiết tài liệu, chia sẻ tài liệu, lấy lịch sử chỉnh sửa và gọi undo/redo.

Các API liên quan trực tiếp đến giao diện cộng tác gồm:

- GET /api/documents: hiển thị dashboard tài liệu
- GET /api/documents/{document_id}: lấy nội dung ban đầu, version, vector_clock và role
- POST /api/documents/{document_id}/share: chia sẻ tài liệu
- DELETE /api/documents/{document_id}/permissions/{permission_id}: xóa quyền user
- GET /api/documents/{document_id}/history: hiển thị lịch sử chỉnh sửa
- POST /api/documents/{document_id}/undo: undo cộng tác
- POST /api/documents/{document_id}/redo: redo cộng tác

3.2. WebSocket Realtime Protocol

Khi người dùng mở editor, frontend tạo kết nối WebSocket tới:

- /ws/documents/{document_id}?token=<jwt_token>

Token được truyền qua query string để backend xác thực người dùng trước khi chấp nhận kết nối.

Sau khi kết nối thành công, server gửi message khởi tạo:

```
{
  "type": "init",
  "document_id": "document-id",
  "user": {
    "id": "user-id",
    "email": "user@example.com",
    "username": "User"
  },
  "role": "editor",
  "content_delta": {
    "ops": [
      { "insert": "Nội dung hiện tại\n" }
    ]
  },
  "version": 5,
  "vector_clock": {
    "user-1": 2,
    "server": 5
  },
  "users": []
}
```

Message này giúp client đồng bộ trạng thái ban đầu của tài liệu.

Ngoài các message phục vụ đồng bộ nội dung, frontend còn xử lý các message realtime liên quan đến quyền truy cập:

- role_changed: Server gửi khi quyền của user trong tài liệu bị thay đổi. Frontend nhận role mới và enable/disable editor tương ứng.

```
{
  "type": "role_changed",
  "document_id": "document-id",
  "role": "viewer"
}
```

- access_removed: Server gửi khi user bị xóa quyền khỏi tài liệu. Frontend hiển thị thông báo và điều hướng user ra khỏi editor.

```
{
  "type": "access_removed",
  "document_id": "document-id",
  "message": "Your access was removed"
}
```

- document_deleted: Server gửi khi tài liệu bị xóa trong lúc user đang mở. Frontend đóng editor và điều hướng về dashboard.

3.3. Message Operation Từ Client

Khi người dùng chỉnh sửa, client gửi message WebSocket loại operation:

```
{
  "type": "operation",
  "operation_delta": {
    "ops": [
      { "retain": 5 },
      { "insert": "abc" }
    ]
  },
  "base_version": 5,
  "vector_clock": {
    "user-1": 3,
    "server": 5
  },
  "client_id": "user-1",
  "client_op_id": "user-1-1710000000-a1b2c3"
}
```

Ý nghĩa:

- operation_delta: thao tác chỉnh sửa ở dạng Quill Delta.
- base_version: phiên bản tài liệu mà client đang dựa vào khi chỉnh sửa.
- vector_clock: đồng hồ vector phía client.
- client_id: định danh client gửi thao tác.

- `client_op_id`: mã định danh duy nhất cho operation do client tạo. Trường này giúp frontend và server nhận biết operation đã được xử lý hay chưa, đặc biệt trong trường hợp reconnect hoặc retry gửi lại operation.

Backend không áp dụng operation ngay, mà chuyển tiếp vào RabbitMQ để worker xử lý.

3.4. Message Operation Đã Được Áp Dụng

Sau khi worker xử lý xong, server broadcast message `operation_applied`:

```
{
  "type": "operation_applied",
  "document_id": "document-id",
  "user_id": "user-1",
  "username": "User",
  "operation_id": "operation-id",
  "client_op_id": "user-1-1710000000-a1b2c3",
  "operation_delta": {
    "ops": [
      { "retain": 5 },
      { "insert": "abc" }
    ]
  },
  "base_version": 5,
  "server_version": 6,
  "vector_clock": {
    "user-1": 3,
    "server": 6
  },
  "causal_relation": "before"
}
```

Khi frontend nhận `operation_applied`, `client_op_id` được dùng để đối chiếu với pending queue. Nếu operation này là thao tác local đã gửi trước đó, frontend xóa operation khỏi queue và cập nhật trạng thái đồng bộ.

3.5. Message Presence

Khi có người vào hoặc rời tài liệu, server gửi message presence:

```
{
  "type": "presence",
  "document_id": "document-id",
  "users": [
    {
      "user_id": "user-1",
      "username": "Alice",
      "role": "owner",
      "color": "#1a73e8",
      "cursor": {
        "index": 10,
        "length": 0
      }
    }
  ]
}
```

Message này dùng để hiển thị danh sách người đang online trong tài liệu.

3.6. Message Cursor

Khi người dùng di chuyển con trỏ, client gửi:

```
{
  "type": "cursor",
  "cursor": {
    "index": 12,
    "length": 0
  }
}
```

- index: vị trí con trỏ trong tài liệu
- length: độ dài vùng chọn
- Nếu length = 0, user chỉ đang đặt con trỏ.
- Nếu length > 0, user đang bôi chọn một vùng văn bản.

Server broadcast lại cho các client khác:

```
{
  "type": "cursor",
  "document_id": "document-id",
  "user_id": "user-1",
  "username": "Alice",
  "color": "#1a73e8",
  "cursor": {
    "index": 12,
    "length": 5
  }
}
```

Chức năng này phục vụ việc mở rộng hiển thị con trỏ realtime theo từng người dùng.

3.7. Ping/Pong

Client có thể gửi message kiểm tra kết nối:

```
{
  "type": "ping"
}
```

Server phản hồi:

```
{
  "type": "pong"
}
```

3.8. RabbitMQ Internal Messages

Bên trong backend, RabbitMQ được dùng để trao đổi giữa Gateway và Worker.

Gateway publish operation vào exchange:

- document.ops.exchange

Queue nhận operation:

- document.ops.queue

Worker xử lý xong thì publish event vào fanout exchange:

- document.events.exchange

Sau đó backend nhận event và broadcast qua WebSocket.

THỬ NGHIỆM VÀ ĐÁNH GIÁ

4.1 Mục tiêu thử nghiệm

Kiểm tra hệ thống ở các nhóm chính: xác thực, quản lý tài liệu, phân quyền, chỉnh sửa realtime, xử lý xung đột, offline/reconnect, undo/redo, lưu trữ Redis/PostgreSQL, RabbitMQ/Worker và tính nhất quán dữ liệu.

4.2 Môi trường thử nghiệm

- Frontend: React + ReactQuill
- Backend: FastAPI
- Database: PostgreSQL
- Cache: Redis
- Message Broker: RabbitMQ
- Worker: OT Worker
- Triển khai: Docker Compose
- Trình duyệt: Chrome tab thường + tab ẩn danh để giả lập nhiều user

4.3 Đăng Ký Và Đăng Nhập

Kịch bản	Cách thử	Kết quả mong đợi
Đăng ký tài khoản mới	Nhập email, username, password hợp lệ	Tạo user thành công, trả JWT
Đăng ký email trùng	Dùng lại email đã tồn tại	Server trả lỗi không cho đăng ký
Đăng nhập đúng	Nhập email/password đúng	Trả access token, vào dashboard
Gọi API không token	Xóa token rồi gọi /api/documents	Server trả 401 Unauthorized

4.4 Quản lý tài liệu

Kịch bản	Cách thử	Kết quả mong đợi
Tạo tài liệu	User bấm tạo document	Document được lưu DB, version = 0
Xem danh sách	Click document	Load title, content_delta, version, vector_clock
Đổi tên tài liệu	Sửa title rồi blur	DB cập nhật title

Xóa tài liệu	Owner bấm delete	Document bị xóa, Redis snapshot cũng bị xóa
--------------	------------------	---

4.5 Chức năng phân quyền

Kịch bản	Cách thử	Kết quả mong đợi
Owner share editor	User A share tài liệu cho User B quyền editor	User B thấy document và sửa được
Owner share viewer	Share quyền viewer	User B mở được nhưng editor bị disable
Viewer cố sửa	Viewer gửi operation	Backend trả lỗi Viewer cannot edit
Đổi editor thành viewer	Owner đổi role khi user đang mở tài liệu	User nhận role_changed, editor bị disable realtime
Thu hồi quyền	Owner xóa permission	User bị gửi access_removed, WebSocket đóng
Đổi editor thành viewer	Owner đổi role khi user đang mở tài liệu	User nhận role_changed, editor bị disable realtime
Thu hồi quyền	Owner xóa permission	User bị gửi access_removed, WebSocket đóng
User không có quyền mở link	Dùng URL document không được share	Server trả 403 hoặc WebSocket close

4.6. WebSocket Và Presence

Kịch bản	Cách thử	Kết quả mong đợi
Kết nối WebSocket hợp lệ	Mở editor với token đúng	Server gửi message init
Token sai/hết hạn	Sửa token hoặc xóa token	WebSocket bị từ chối

Nhiều user mở cùng tài liệu	Mở bằng 2 trình duyệt	Presence hiển thị nhiều user
User thoát tài liệu	Đóng tab một user	Presence cập nhật lại
Ping/Pong	Client gửi {type:"ping"}	Server trả {type:"pong"}

4.7. Chỉnh Sửa Realtime Cơ Bản

Kịch bản	Cách thử	Kết quả mong đợi
Một user gõ	User A nhập chữ	Worker xử lý, version tăng, nội dung lưu DB/Redis
Hai user cùng mở	User A gõ, User B quan sát	User B nhận operation_applied gần realtime
Nội dung hội tụ	Sau nhiều lần sửa, reload cả hai tab	Hai client hiển thị cùng nội dung
Version tăng đúng	Mỗi operation thành công	version tăng thêm 1
Vector clock cập nhật	Sau mỗi operation	server tick tăng, user clock được merge

4.8. Xử Lý Xung Đột Đồng Thời

Kịch bản	Cách thử	Kết quả mong đợi
Hai user sửa khác vị trí	A sửa đầu câu, B sửa cuối câu	Cả hai thay đổi đều còn
Hai user sửa gần cùng lúc	A và B gõ nhanh cùng thời điểm	Worker transform delta, document không bị ghi đè
Operation có base_version cũ	Cho B offline/chậm, A sửa trước, B gửi sau	Worker transform theo history
Clock before	Client gửi operation dựa trên server version cũ	Operation vẫn được xử lý, log before

Clock after	Client tick clock user trước khi gửi	Operation được xử lý, log after
Clock concurrent	Hai user có thay đổi độc lập	Operation được xử lý, log concurrent
Clock equal	Client và server cùng clock	Operation apply bình thường

4.9. Chống Trùng Operation

Kịch bản	Cách thử	Kết quả mong đợi
Gửi lại cùng client_op_id	Simulate reconnect hoặc gửi lại message cũ	Worker không apply lần hai
Kiểm tra nội dung	Gõ a, gửi duplicate	Nội dung chỉ có a, không thành aa
Kiểm tra log	Xem document_operations	Không có operation trùng gây tăng version sai
ACK duplicate	Worker trả duplicate_ack	Client xóa pending operation

4.10. Offline Và Reconnect

Kịch bản	Cách thử	Kết quả mong đợi
Mất mạng/WebSocket đóng	Tắt backend tạm hoặc ngắt network browser	Frontend chuyển Offline
Gõ khi offline	User tiếp tục nhập	Operation lưu localStorage/pending queue
Kết nối lại	Bật backend/network lại	Frontend flush pending queue
Không mất dữ liệu	Sau reconnect	Nội dung offline được đồng bộ lên server
Không gửi trùng	Reconnect nhiều lần	client_op_id chống duplicate

4.11. Redis Và PostgreSQL

Kịch bản	Cách thử	Kết quả mong đợi
----------	----------	------------------

Đọc snapshot từ Redis	Mở tài liệu sau khi đã sửa	Backend/worker lấy snapshot nhanh từ Redis
Redis mất cache	Xóa Redis data hoặc restart Redis	Hệ thống đọc lại từ PostgreSQL
PostgreSQL lưu bền vững	Restart toàn bộ app, giữ volume DB	Document vẫn còn
Redis cập nhật sau operation	Gỡ mới	Redis snapshot có content/version/vector_clock mới

4.12. RabbitMQ Và Worker

Kịch bản	Cách thử	Kết quả mong đợi
Backend publish operation	User gõ	Message vào document.ops.queue
Worker consume operation	Worker đang chạy	Queue giảm, DB được cập nhật
Worker publish event	Sau xử lý	Event vào document.events.exchange
Backend broadcast event	Backend nhận event	Client nhận operation_applied
Worker tạm dừng	Stop worker rồi gõ	Message tồn trong queue
Worker chạy lại	Start worker	Message được xử lý tiếp

4.13. History, Version, Rollback, Undo/Redo

Kịch bản	Cách thử	Kết quả mong đợi
Xem history	Mở HistoryPanel	Hiển thị operation log
Xem versions	Mở danh sách version	Hiển thị snapshot theo version
Rollback	Chọn version cũ	Tạo version mới với content cũ
Undo	Bấm Ctrl+Z hoặc nút Undo	Server tạo version mới sau undo

Redo	Bấm Ctrl+Y hoặc nút Redo	Server khôi phục version tiếp theo
Đồng bộ sau undo/redo	Hai user cùng mở	Cả hai nhận snapshot mới

4.14. Đánh Giá Hệ Thống

Tính đúng đắn

- User chỉ truy cập/sửa tài liệu theo đúng quyền.
- Mỗi operation hợp lệ làm version tăng đúng 1.
- Duplicate operation không bị apply hai lần.

Tính nhất quán

- Các client có thể lệch tạm thời trong lúc worker xử lý.
- Sau khi nhận operation_applied, tất cả hội tụ về cùng content_delta.

Tính chịu lỗi

- Mất WebSocket không làm mất thao tác nhờ pending queue.
- Redis mất cache vẫn phục hồi từ PostgreSQL.
- Worker dừng tạm thời thì RabbitMQ giữ operation chờ xử lý.

Tính realtime

- Với 2-5 client thử nghiệm, thay đổi được broadcast gần như tức thời.
- Presence và cursor cập nhật theo vòng đời WebSocket.

Tính mở rộng

- Backend gateway tách khỏi worker qua RabbitMQ.
- Worker có thể scale theo queue.
- PostgreSQL lưu bền vững, Redis tăng tốc đọc snapshot.